NLP — Unit 5 Practice Programs

Chatbot | Question Answering | Summarization | Machine Translation

---

> **Tip:** Runtime → Change runtime type → **GPU** for faster loading.

Cell 1 — Install Dependencies

> Pinned to `transformers==4.44.2` for full pipeline compatibility.

```
!pip install -q transformers==4.44.2 torch sentencepiece
print('All libraries installed successfully!')
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.7/43.7 kB 2.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.5/9.5 MB 40.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 566.4/566.4 kB 18.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.6/3.6 MB 56.2 MB/s eta 0:00:00
All libraries installed successfully!
```

Cell 2 — Imports

```
from transformers import pipeline
import warnings, torch
warnings.filterwarnings('ignore')
device = 0 if torch.cuda.is_available() else -1
print('GPU available:', torch.cuda.is_available())
print('Imports done!')
```

```
GPU available: True
Imports done!
```

Program 1 — Conversational Chatbot

Model: `microsoft/DialoGPT-small`
Maintains conversation history across turns.

```
print('Loading chatbot model...')
chatbot = pipeline('text-generation', model='microsoft/DialoGPT-small', device=devi
print('Chatbot model ready!')
```

```
Loading chatbot model...
```

config.json: 100% 641/641 [00:00<00:00, 24.6kB/s]

model.safetensors: 100% 351M/351M [00:03<00:00, 290MB/s]

generation_config.json: 100% 124/124 [00:00<00:00, 7.62kB/s]

tokenizer_config.json: 100% 614/614 [00:00<00:00, 20.4kB/s]

vocab.json: 1.04M/? [00:00<00:00, 15.8MB/s]

merges.txt: 456k/? [00:00<00:00, 8.57MB/s]

```
Chatbot model ready!
```

```
def chat_once(user_input, history=''):
    prompt = history + 'User: ' + user_input + '\nBot:'
    out = chatbot(prompt, max_new_tokens=80, do_sample=True,
                  temperature=0.75, pad_token_id=50256)
    reply = out[0]['generated_text'].split('Bot:')[-1].strip()
    reply = reply.split('User:')[0].strip()
    return reply

def multi_turn_chat(turns):
    history = ''
    print('=' * 60)
    print('  CHATBOT SESSION')
    print('=' * 60)
    for user_msg in turns:
        reply = chat_once(user_msg, history)
        print('  You :', user_msg)
        print('  Bot :', reply)
        print('-' * 60)
        history += 'User: ' + user_msg + '\nBot: ' + reply + '\n'

conversation = [
    'Hi! What is Natural Language Processing?',
    'Can you give me an example?',
    'What are transformers in NLP?',
    'Thank you! Goodbye.',
]
multi_turn_chat(conversation)
```

```
============================================================
  CHATBOT SESSION
============================================================
  You : Hi! What is Natural Language Processing?
  Bot : natural language processing
------------------------------------------------------------
  You : Can you give me an example?
  Bot : Can you explain the function?
------------------------------------------------------------
  You : What are transformers in NLP?
  Bot : What is that?
------------------------------------------------------------
  You : Thank you! Goodbye.
  Bot : Goodbye User.
------------------------------------------------------------
```

Program 2 — Question Answering

Model: `deepset/roberta-base-squad2`

Extracts exact answer spans from a context passage.

```
print('Loading QA model...')
qa = pipeline('question-answering', model='deepset/roberta-base-squad2')
print('QA model ready!')
```

```
Loading QA model...
config.json: 100%                                   571/571 [00:00<00:00, 40.0kB/s]
model.safetensors: 100%                             496M/496M [00:06<00:00, 57.6MB/s]
tokenizer_config.json: 100%                         79.0/79.0 [00:00<00:00, 7.55kB/s]
vocab.json:      899k/? [00:00<00:00, 38.5MB/s]
merges.txt:      456k/? [00:00<00:00, 26.8MB/s]
special_tokens_map.json: 100%                       772/772 [00:00<00:00, 84.2kB/s]
Hardware accelerator e.g. GPU is available in the environment, but no `device` argum
QA model ready!
```

```
def answer_questions(label, context, questions):
    print('Context:', label)
    print('{:<48} {:<30} {:>10}'.format('Question', 'Answer', 'Confidence'))
    print('-' * 90)
    for q in questions:
        res = qa(question=q, context=context)
        print('{:<48} {:<30} {:>10.2%}'.format(q, res['answer'], res['score']))
    print()

answer_questions(
    'India',
    ('India, officially the Republic of India, is a country in South Asia. '
     'New Delhi is the capital of India. India is the most populous country '
     'in the world with over 1.4 billion people. It is known for its diverse '
     'culture, rich history, and contributions to mathematics and science. '
     'The national language is Hindi, and English is widely spoken.'),
    [
        'What is the capital of India?',
        'How many people live in India?',
        'What is the national language of India?',
    ]
)

answer_questions(
    'Python Programming',
    ('The Python programming language was created by Guido van Rossum '
     'and first released in 1991. Python emphasises code readability '
     'and supports multiple programming paradigms. It is widely used '
     'in data science, web development, and artificial intelligence.'),
    [
        'Who created Python?',
        'When was Python released?',
        'What is Python used for?',
    ]
)
```

```
Context: India
Question                                           Answer                         Conf
------------------------------------------------------------------------------------
What is the capital of India?                      New Delhi
```

```
How many people live in India?                  1.4 billion
What is the national language of India?         Hindi

Context: Python Programming
Question                                        Answer                        Conf
--------------------------------------------------------------------------------------
Who created Python?                             Guido van Rossum
When was Python released?                       1991
What is Python used for?                        data science, web development, and
```

Program 3 — Text Summarization

Model: `facebook/bart-large-cnn`
Fix: Uses `text2text-generation` task (compatible with all transformers versions).

```
print('Loading summarization model...')
from transformers import BartForConditionalGeneration, BartTokenizer

model_name = 'facebook/bart-large-cnn'
tokenizer  = BartTokenizer.from_pretrained(model_name)
model      = BartForConditionalGeneration.from_pretrained(model_name)

if torch.cuda.is_available():
    model = model.to('cuda')

print('Summarizer ready!')
```

```
Loading summarization model...
vocab.json:       899k/? [00:00<00:00, 41.2MB/s]
merges.txt:       456k/? [00:00<00:00, 25.7MB/s]
tokenizer.json:       1.36M/? [00:00<00:00, 49.3MB/s]
config.json:       1.58k/? [00:00<00:00, 169kB/s]
model.safetensors: 100%                                    1.63G/1.63G [00:08<00:00, 284MB/s]
generation_config.json: 100%                                  363/363 [00:00<00:00, 20.4kB/s]
Summarizer ready!
```

```
def summarize(title, text, max_len=120, min_len=30):
    dev = 'cuda' if torch.cuda.is_available() else 'cpu'
    inputs = tokenizer(text.strip(), return_tensors='pt',
                       max_length=1024, truncation=True).to(dev)
    ids     = model.generate(inputs['input_ids'],
                             max_length=max_len,
                             min_length=min_len,
                             num_beams=4,
                             length_penalty=2.0,
                             early_stopping=True)
    summary = tokenizer.decode(ids[0], skip_special_tokens=True)
    orig_w  = len(text.split())
    summ_w  = len(summary.split())
    ratio   = (1 - summ_w / orig_w) * 100
    print('Topic      :', title)
    print('Summary    :', summary)
    print('Compression: {} -> {} words  ({:.0f}% reduction)'.format(orig_w, summ_w,
    print()
```

```
texts = {
    'Artificial Intelligence': (
        'Artificial Intelligence (AI) is a branch of computer science that focuses
        'on building systems capable of performing tasks that normally require huma
        'intelligence. These tasks include visual perception, speech recognition, '
        'decision-making, and language translation. Machine learning, a subset of A
        'enables systems to learn from data without being explicitly programmed. '
        'Deep learning uses neural networks with many layers to achieve breakthroug
        'in image recognition, NLP, and autonomous driving. AI is now embedded in '
        'everyday products such as search engines, recommendation systems, and '
        'virtual assistants like Siri and Alexa.'
    ),
    'Climate Change': (
        'Climate change refers to long-term shifts in temperatures and weather patt
        'While some changes are natural, scientific evidence shows that human activ
        'especially the burning of fossil fuels have been the main driver of climat
        'change since the 1800s. The consequences include rising sea levels, more '
        'frequent extreme weather events, and disruptions to ecosystems worldwide.
        'International agreements such as the Paris Accord aim to limit global warm
        'by reducing greenhouse gas emissions globally.'
    ),
}

for title, text in texts.items():
    summarize(title, text)
```

```
Topic      : Artificial Intelligence
Summary    : Artificial Intelligence (AI) is a branch of computer science that focus
Compression: 88 -> 50 words  (43% reduction)

Topic      : Climate Change
Summary    : Climate change refers to long-term shifts in temperatures and weather p
Compression: 73 -> 45 words  (38% reduction)
```

Program 4 — Machine Translation

Models: `Helsinki-NLP/opus-mt` family

Translates English into French, Hindi, and German.

```
print('Loading 3 translation models...')
translators = {
    'French' : pipeline('translation_en_to_fr', model='Helsinki-NLP/opus-mt-en-fr')
    'Hindi'  : pipeline('translation',          model='Helsinki-NLP/opus-mt-en-hi')
    'German' : pipeline('translation_en_to_de', model='Helsinki-NLP/opus-mt-en-de')
}
print('All translation models ready!')
```

```
Loading 3 translation models...
config.json:        1.42k/? [00:00<00:00, 111kB/s]
pytorch_model.bin: 100%                                    301M/301M [00:02<00:00, 157MB/s]
generation_config.json: 100%                               293/293 [00:00<00:00, 29.3kB/s]
tokenizer_config.json: 100%                                42.0/42.0 [00:00<00:00, 4.53kB/s]
source.spm: 100%                                           778k/778k [00:00<00:00, 68.5MB/s]
target.spm: 100%                                           802k/802k [00:00<00:00, 58.3MB/s]
vocab.json:         1.34M/? [00:00<00:00, 54.3MB/s]
Hardware accelerator e.g. GPU is available in the environment, but no `device` argum
config.json:        1.39k/? [00:00<00:00, 145kB/s]
pytorch_model.bin: 100%                                    306M/306M [00:02<00:00, 730MB/s]
generation_config.json: 100%                               293/293 [00:00<00:00, 21.7kB/s]
tokenizer_config.json: 100%                                44.0/44.0 [00:00<00:00, 4.95kB/s]
source.spm: 100%                                           812k/812k [00:00<00:00, 43.6MB/s]
target.spm: 100%                                           1.07M/1.07M [00:00<00:00, 57.7MB/s]
vocab.json:         2.10M/? [00:00<00:00, 64.6MB/s]
Hardware accelerator e.g. GPU is available in the environment, but no `device` argum
config.json:        1.33k/? [00:00<00:00, 65.8kB/s]
pytorch_model.bin: 100%                                    298M/298M [00:02<00:00, 159MB/s]
generation_config.json: 100%                               293/293 [00:00<00:00, 31.3kB/s]
tokenizer_config.json: 100%                                42.0/42.0 [00:00<00:00, 5.27kB/s]
source.spm: 100%                                           768k/768k [00:00<00:00, 58.3MB/s]
target.spm: 100%                                           797k/797k [00:00<00:00, 62.3MB/s]
vocab.json:         1.27M/? [00:00<00:00, 35.8MB/s]
Hardware accelerator e.g. GPU is available in the environment, but no `device` argum
All translation models ready!
```

```
sentences = [
    'Hello, how are you?',
    'Natural Language Processing is fascinating.',
    'I love learning new things every day.',
    'Artificial Intelligence is the future.',
]

for sentence in sentences:
    print('EN :', sentence)
    for lang, model in translators.items():
        result = model(sentence)[0]['translation_text']
        print('{} : {}'.format(lang[:2].upper(), result))
    print()
```

```
EN : Hello, how are you?
FR : Bonjour, comment allez-vous ?
HI : हैलो, तुम कैसे हो?
GE : Hallo, wie geht's?

EN : Natural Language Processing is fascinating.
FR : Le traitement du langage naturel est fascinant.
```

```
HI : स्वाभाविक भाषा अनुवाद बहुत ही दिलचस्प है ।
GE : Natürliche Sprachverarbeitung ist faszinierend.

EN : I love learning new things every day.
FR : J'adore apprendre de nouvelles choses chaque jour.
HI : मुझे हर दिन नयी – नयी बातें सीखने से बहुत खुशी मिलती है ।
GE : Ich liebe es, jeden Tag neue Dinge zu lernen.

EN : Artificial Intelligence is the future.
FR : L'intelligence artificielle est l'avenir.
HI : कलावादी अधिकार भविष्य है.
GE : Künstliche Intelligenz ist die Zukunft.
```

All Programs Complete!

#	Task	Model	Method
1	Chatbot	DialoGPT-small	`pipeline('text-generation')`